



KVITLI · DESIGN DOCUMENT

Your business dashboard

An account-based business-management web app for freelancers and small businesses — leads, orders, projects, clients, invoices and contracts in one place, with an embeddable order form.

VERSION	1.0
TYPE	Business-management app
TECH	HTML · CSS · JS · Firebase · EmailJS
INTERFACE	Bilingual (HU/EN)

CONTENTS

Table of contents

01	Overview
02	Goals & audience
03	Tabs & features
04	Workflow
05	Embeddable form
06	Architecture & tech
07	Data model
08	Storage & security
09	Design system
10	File structure
11	Setup
12	Future work

Overview

Kvitli is an account-based business-management web app for freelancers and small businesses. In one place it handles incoming leads, orders, projects, clients, invoices and contracts, plus income and expenses.

Data is stored per account in the cloud (Firebase). No build step, no framework: plain HTML, CSS and JavaScript. The interface is bilingual (Hungarian/English) and uses the shared "Studio" design with a blue primary and green accent, in light/dark/auto.

Philosophy: no server of your own. Static hosting + Firebase (Auth + Firestore) + EmailJS — card-free, maintenance-free operation with cross-device sync.

Goals & audience

Audience

Freelancers and small businesses who want to follow their work from first enquiry to issued invoice in one place — without separate spreadsheets.

Goal

A single, always-current surface for the whole client and finance flow, which also pulls in orders from the website automatically.

Kvitli replaces the paperwork: an enquiry becomes a client, an order a project, a project an invoice — every step tracked and exportable.

Tabs & features

TAB	FUNCTION
Overview	This year's revenue, ongoing/total projects, upcoming deadlines, charts.
Orders	Pricing incoming leads, sending offers and contracts.
Projects	Jobs with status (in progress, test, live, paid) and deadlines.
Clients	An automatic client base built from leads and projects.
Invoices	Issuing invoices, paid/overdue state, printable PDF.
Contract templates	Industry-specific contracts, signed online, as PDF.
Income / Expense	An itemised finance log with categories, monthly totals.
Form	Generator for an embeddable order form (services, fields, prices).
Account	Business data, appearance, language, export/import, OWNER_UID.

Workflow

Kvitli guides a clear flow: lead → offer → contract → order → project → invoice.

Offer You send the offer by e-mail with a PDF download and one-click accept; the status updates automatically.	Contract The client signs online with a drawn signature; it is assembled from your business data and the accepted offer.	Invoice Finished work becomes a printable invoice with paid/overdue tracking.
--	--	---

All e-mails (offer, confirmation) are sent via EmailJS from the user's browser, with no separate mail server.

Embeddable form

A dedicated tab generates an embeddable order form: your settings assemble into code you paste onto your site once.

The form config is written to the Firestore `form_configs/<uid>` document; the embedded form reads the services, fields and prices from there, and writes a submitted order under `inbox/<uid>/items` — so it appears live on the Orders tab with no manual entry.

The public Firebase keys are browser identifiers; the form needs the account's `OWNER_UID`, which the Account tab shows in the snippet to paste.

Architecture & tech

Client

Vanilla HTML/CSS/JS, no build step. Auth + app core, numeric auto-fit, bilingual, light/dark theme.

Cloud

Firebase Authentication (email/password) + Cloud Firestore. Static hosting for the site; e-mails via EmailJS.

Script load order: Firebase SDK → firebase-store.js → script.js (+ leads.js, contract.js, fit-text.js). Charts are drawn on a custom canvas, no external library.

Data model

Each user's full state lives in a single Firestore document (`vaults/<uid>`); the embedded form uses two extra collections.

```
// vaults/<uid> - main parts of the state
{
  orders: [ ... ], leads: { ... }, invoices: [ ... ],
  bizIncome: [ ... ], bizExpense: [ ... ],
  formConfig: { services, fields, ... },
  sellerInfo, profile, settings, ...
}

form_configs/<uid>    // published form config
inbox/<uid>/items     // incoming orders
```

The data layer (`firebase-store.js`) makes the Firestore calls; it loads on sign-in and saves on change.

Storage & security

Private data: `vaults/<uid>` is read/write only for the signed-in owner.

Controlled public surface: `form_configs` is readable (so the embedded form works), while `inbox` is write-only from outside — incoming orders can only be read by the owner.

Public keys: the Firebase keys are browser-public identifiers; no secret key reaches the client.

The Firestore rules live in the `firebase/` folder and deploy via the CLI.

Design system

A modern, clean "Studio" design — a blue primary with a green accent, rounded cards, light/dark theme.

COLOUR PALETTE

	Blue (primary)	#2378be
	Deep blue	#1a5f9e
	Green (accent)	#6ba83a
	Light base	#eef2f7
	Ink	#16212c
	Line	#d3e0ec

Heading / UI Inter · 400–800	Mono IBM Plex Mono	Category colours tuned to the blue-green family
--	------------------------------	---

Components: blue accent on buttons, green on positive states, red on overdue/danger cues; large figures are sized by fit-text.js.

File structure

```
04_Kvitli/  
├─ index.html, ajanlat.html, szerzodes.html  
├─ css/style.css  
├─ js/ (script, leads, contract, firebase-store, fit-text, theme)  
├─ img/ (logo, favicon)  
├─ email-sablonok/ (EmailJS templates)  
├─ firebase/ (config + rules)  
└─ dokumentumok/ (design doc, setup)
```

Setup

- Firebase: project, Authentication (email/password), Firestore; fill the config in index.html and deploy the rules from the firebase/ folder.
- EmailJS: configure the service + templates (see BEALLITAS-EmailJS.txt).
- Publish: upload the files to static hosting (GitHub Pages).
- Embed: paste the code generated on the Form tab into your own website.

Future work

- Automatic invoicing and payment reminders.
- More contract templates and customisable fields.
- Reports and export for accounting.

Thanks to the modular, framework-free structure, extensions can be added gradually without disrupting operation.